



Norwegian University of
Science and Technology

Practical Anonymous Communication with Homomorphic Encryption

Angelo, Imre

Submission date: July 2026

Main supervisor: Associate Professor Park, Jeongeun, NTNU

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Many anonymous communication systems have been proposed and implemented with the goal of allowing users to exchange messages over a network without revealing who is communicating with whom. Most of these designs rely on the *threshold model* to provide anonymity, wherein some components of their infrastructure must be behaving honestly. Systems with stronger anonymity guarantees generally suffer from poor scalability or are impractical to instantiate.

This thesis examines a new construction called a (Somewhat) Practical Anonymous Router (sPAR), which addresses the trust and practicality concerns of existing systems. The construction relies only on well-established Fully Homomorphic Encryption (FHE) schemes, making it realistically implementable. While sPAR presents a promising new avenue for constructing anonymous communication systems, it remains a theoretical construction. A concrete implementation and a deeper investigation is therefore needed to evaluate its practical performance and viability in larger networks.

In an effort to investigate sPAR's viability, this thesis aims to implement the scheme and conduct a structured evaluation of its performance with respect to the number of participants in the system.

Research Questions:

- What is the computational cost of sPAR, and how does it compare to the theoretical complexity described in the original paper?
- How does the performance of sPAR scale with the number of participants, and at what point does it become impractical?

Approved: 2026-04-01 – Associate Professor Park, Jeongeun, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Sammendrag

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures	ix
List of Tables	xi
List of Algorithms	xiii
1 Background	1
2 Related Works	3
2.1 Fully Homomorphic Encryption	4
2.1.1 The BGV scheme	6
2.2 Residue Number Systems	7
2.2.1 Number Theoretic Transform	8
2.2.2 Base Conversion	9
2.3 Key-Switching	12
2.3.1 Brakerski-Vaikuntanathan	12
2.3.2 RGSW	14
2.4 (Somewhat) Practical Anonymous Router	17
3 Methodology	19
3.1 Implementation Details of the Core Library	20
3.1.1 Architecture	20
3.1.2 The BV Instantiation	21
3.2 Implementation Details of sPAR	25
3.2.1 Deviations from the Specification	25
3.2.2 Procedural Design	25
3.2.3 Server Write Step	25
3.3 Experimental Setup	27
3.3.1 Protocol Simulation	27
3.3.2 Unit Tests and Benchmarks	27
3.3.3 Noise Measurement	28
4 Results and Discussion	29

4.1	Parameters	30
4.2	Noise Analysis	31
4.2.1	Core Library	31
4.2.2	sPAR	32
4.3	Performance Benchmarks	33
4.3.1	Core Library	33
4.3.2	sPAR	34
4.3.3	Multi-Threaded Performance	34
4.4	Discussions	34
4.4.1	Theoretical Hybrid Improvement	35
5	Conclusion and Future Work	37
5.1	Conclusion	38
5.2	Future Work	38
	References	39
	Appendices	
A	Number Theory	41
A.1	Chinese Remaineder Theorem	41
A.2	Number Theoretic Transform	41

List of Figures

4.1	RNS-BV noise growth in chained external products	31
4.2	RNS-BV noise growth in chained internal products TODO: Overlay graph of using accumulated result on rhs	32
4.3	Server::Write	33

List of Tables

3.1	Operations provided by <code>IExtendedContext</code>	21
3.2	Bitwise Operators	24
4.1	Base parameters for BV-RNS	30
4.2	RGSW Operations (todo: update numbers)	33
4.3	One round of the full sPAR protocol, not adjusted to per user except Write, 6 threads	34

List of Algorithms

2.1	Fast Base Extension	10
2.2	SCALE(x, P)	10
3.1	Parallel RGSW-BV Encryption	23
3.2	Center	24
3.3	Server::Write	26
4.1	Hybrid-RGSW Encryption	35
4.2	RLWE $\square$ Hybrid-RGSW	36
4.3	Modified	36

Chapter 1

Background

Anonymous communication aims to enable users to exchange messages over a network without revealing who is communicating with whom, thereby protecting both sender and receiver identities. Many such systems have been proposed and implemented over the years, such as Tor, I2P, and mixnets. Most of these designs depend on the *threshold model* to provide anonymity, requiring at least some components of their infrastructure to behave honestly. In practice, this assumption is increasingly strained by large-scale surveillance, correlation attacks, and the growing centralization of internet infrastructure [SEV+15].

Systems with stronger anonymity guarantees based on Multi-Party Communication (MPC) exist. However, these suffer from poor scalability due to their high interactivity, making them impractical for large-scale networks. A Dining Cryptographers network (DC-net), for example, can provide anonymity even if there are no trusted components in the network infrastructure. Doing so requires every participant to interact with every other participant, for every round of messages sent.

Recently, a new line of research has proposed fundamentally different designs, aiming to offer strong anonymity even in the presence of compromised or adversarial components in the network actively attempting to break anonymity. The first such design was the Non-Interactive Anonymous Router (NIAR) [SW21], which proposing using Multi-Client Functional Encryption (MCFE) to allow a single, untrusted router to shuffle or permute messages from a set of clients *obliviously*; without learning anything about the permutation. Thereby making it infeasible for the router to link any of the messages to its sender.

NIAR was a breakthrough in anonymous communication, not only showing the theoretical possibility of designing anonymous routers, but also that such routers can be *non-interactive*: clients only send one message to the router and do not need to perform any additional interaction to achieve anonymity. Using a centralized (anonymous) router in this way yields similar benefits to existing MPC-based systems,

2 1. BACKGROUND

without needing any interactions between clients. However, there are a number of problems that makes NIAR impractical, if not impossible, to implement in practice.

1. The cryptographic components required by NIAR are extremely computationally expensive or not known to be implementable in practice.
2. Achieving anonymity for recipients assumes the existence of a very strong primitive called *indistinguishability obfuscation with sub-exponential security*: guaranteeing that no efficient adversary can distinguish between the obfuscations of two functionally equivalent circuits with more than a sub-exponentially small advantage. The practicality of such a construction is unknown.
3. Reusing the same setup across multiple rounds makes messages from the same sender linkable, unless the expensive setup phase is rerun for every round.

These problems were highlighted in sPAR

- Anonymous communication
- Current solutions
- Problems with current solutions
- FHE as a new approach
- FHE 'holy grail' of privacy-preserving communication
- sPAR
- RSA is multiplicatively homomorphic, but (follow-up) paper theorized a *fully* homomorphic scheme may be possible [Gen09; RAD78]

Structure of the thesis

- Building blocks and related works; FHE, gadget decomposition, TFHE, etc.
- Explanation of the code
- Experimental setup i.e. how the numbers where found
- Results, benchmarks, noise analysis
- Future work

Chapter 2

Related Works

This section details the fundamental building blocks of the sPAR protocol, and the protocol itself. To achieve a reasonable depth the protocol utilizes the *external product*, a primitive from TFHE bootstrapping that allows homomorphic multiplication of binary digits with near-constant noise bounds — enabling a large number of binary multiplication without consuming a level/bootstrapping. This primitive was ported to BGV-RNS for this thesis, which does not natively allow digit decomposition as normally used by the external product.

2.1 Fully Homomorphic Encryption

Homomorphic Encryption (HE) allows computation directly on encrypted data: an untrusted party can evaluate a function over ciphertexts without learning anything about the underlying plaintexts.

With few exceptions, all of the best performing FHE frameworks [BFV; CKKS; TFHE; BGV11] rely on the hardness of (the ring-variant of) the Learning With Errors (LWE) problem first introduced in [regev:2009].

Definition 2.1. Centered Residue Representation Mathematically the two groups are isomorphic, we are simply renaming $q/2 + 1 \mapsto -q/2$ etc., but in practice this affects noise bounds by keeping $\|\epsilon\| \leq q/2$ instead of $\|\epsilon\| \leq q$.

$$[a]_q = (a + q/2 \bmod q) - q/2 \mapsto [-q/2, q/2) \quad (2.1)$$

$$\mathbb{Z}_q = \{-q/2, -q/2 + 1, \dots, -1, 0, 1, \dots, q/2 - 1\} \quad (2.2)$$

- Concretely state what happens when $2 \nmid q$ for example $\mathbb{Z}_5 = \{-2, -1, 0, 1, 2\}$ is not really $\pm 5/2 = 2.5$

Definition 2.2. RLWE ciphertext $\vec{x} = (b, a) \in R_q^2$ containing a uniformly random *public element* a and the masked element $b = a \cdot s + \Delta m + \epsilon$ derived from a persistent secret key s and randomly sampled noise ϵ , where Δ is the message scaling factor and $m \in R_t$ is the plaintext message being encrypted.

$$\text{RLWE}(m) = (b, a) = (a \cdot s + \Delta m + \epsilon, a) \quad (2.3)$$

$$s \xleftarrow{\$} \{-1, 0, 1\}^N, \quad \epsilon \xleftarrow{\$} \chi, \quad a \xleftarrow{\$} R_q \quad (2.4)$$

An RLWE ciphertext $\vec{x}$ encrypting a message m under the secret key s is denoted $\text{RLWE}_s(m)$ unless the secret key is obvious from context, then $\text{RLWE}(m)$ is sufficient.

Definition 2.3. RLWE Public Key A public key is an encryption of 0 $p = (a \cdot s + \epsilon, a)$ so that encryption using the public key is as simple as adding the encoded message into

Definition 2.4. Encryption

$$\text{ENCRYPT}(m) \mapsto (a \cdot s + \text{ENCODE}(m) + \epsilon, a) \quad (2.5)$$

$$\text{ENCRYPT}(m, pk) = pk + (\text{ENCODE}(m), 0) \quad (2.6)$$

Definition 2.5. Decryption

$$\text{DECRYPT}(x, s) = m \quad (2.7)$$

FHE schemes support homomorphic addition and multiplication, and typically admits one of the two for *free* in the sense that applying the operation directly to the ciphertexts applies it to the underlying plaintexts, without any additional machinery. Schemes based on Ring Learning With Errors (RLWE) have free addition, as seen below:

$$\begin{aligned}
\vec{x} + \vec{y} &= (b_x, a_x) + (b_y, a_y) \\
&= (a_x s + \Delta m_x + \epsilon_x, a_x) + (a_y s + \Delta m_y + \epsilon_y, a_y) \\
&= ((a_x + a_y)s + \Delta(m_x + m_y) + (\epsilon_x + \epsilon_y), a_x + a_y) \\
&\quad \text{Let } a = (a_x + a_y) \text{ and } \epsilon = (\epsilon_x + \epsilon_y) \\
\therefore \vec{x} + \vec{y} &= (as + \Delta(m_x + m_y) + \epsilon, a) = \text{RLWE}_s(m_x + m_y)
\end{aligned}$$

Definition 2.6. RLWE addition

$$+ := \text{RLWE}_s(m_x) + \text{RLWE}_s(m_y) = \text{RLWE}_s(m_x + m_y) \quad (2.8)$$

- The resulting RLWE ciphertext encodes the sum of the original messages, but also has higher noise.
- Managing noise is a central issue in FHE, especially when it comes to multiplication in LWE-based schemes
- Homomorphic multiplication is considerably more complicated and expensive
- We mix BGV with TFHE-style external products to achieve multiplication (of binary digits), and avoid any HE multiplication.

Definition 2.7. Modulus Switching The process of converting a ciphertext $\vec{x} \in R_Q^2$ encrypted under a ciphertext modulus Q into a ciphertext $\vec{x}' \in R_{Q'}^2$, encrypted under a different modulus Q' , while ensuring that both ciphertexts still decrypt to the same message.

Scaling the message by a factor of Q'/Q and rounding to the nearest integer achieves this,

This process is typically the primary tool for noise management in leveled schemes, as scaling down the modulus also scales down the noise, allowing further computation without bootstrapping.

Definition 2.8. Bootstrapping The process of homomorphically evaluating the decryption circuit to reset the noise of a ciphertext, allowing further computation [Gen09]. Generally slow.

Definition 2.9. Keyswitching The process of converting a message to being encrypted under a different secret key.

Definition 2.10. Multi-Party FHE Different methods exist and are being researched. Method used in this thesis: Each party has a secret key and public key.

The full public key is each of the public key shares appended together. Noise is flooded to maintain security. Partial decryption = each client decrypts the ciphertext using their own secret key Final decryption = the server takes each partial decryption and since the decryption is linear can combine them to decrypt the message without any party knowing the (full) secret key.

2.1.1 The BGV scheme

- Introduce scheme [BGV11]
- Justify choice implicitly; strengths
 - SIMD

While most of the theory covered throughout this thesis is scheme-agnostic, the implementation is built on top of the BGV scheme [BGV11], **chosen because** ... which places the message in the *least* significant bits of the ciphertext by encoding the message v as a plaintext m via

$$m = \frac{\tilde{W} \cdot I_n^R \cdot v}{n} \quad (2.9)$$

Notably, $v = 0 \implies m = 0$ and $v = 1 \implies m = 1$, which is important for later.

The message scale is fixed to $\Delta = 1$ **(But some noise/scaling techniques i.e. FLEXIBLEAUTO can change Δ in practice)** and the noise is scaled by the plaintext modulus, $\epsilon = t \cdot \epsilon_{\text{rlwe}}$, giving $b = a \cdot s + m + t\epsilon_{\text{rlwe}}$ in (2.3). In this thesis, each RLWE ciphertext will be expressed as (2.3) to be as general as possible, even though the BGV stuff is implied.

Definition 2.11. Decryption Assuming $\epsilon < Q$, then decryption is correct.

$$\text{DECRYPT}(x, s) = \llbracket b - a \cdot s \rrbracket_t \quad (2.10)$$

Definition 2.12. SIMD Batching If the chosen plaintext modulus t is a prime satisfying the algebraic condition $t \equiv 1 \pmod{2N}$, then BGV supports per-slot SIMD packing [SV11]. This mode converts a plaintext from a polynomial of order $N - 1$ into N independent constants $[c_i]_t$. Homomorphic operations are applied to each slot individually, for example $(1, 2) \cdot (2, 3) = (2, 6)$ instead of $(1, 2) \cdot (2, 3) = (1 + 2x)(2 + 3x) = (2, 7, 6)$.

2.2 Residue Number Systems

In BGV, BFV and CKKS the ciphertext modulus Q is large, much larger than a computer word/register (typically 64 bits on modern computers). Numbers that do not fit inside a register must be handled with multi-precision arithmetic, which is known to be slow due to its serial nature that does not allow parallelization. High-performance instantiations avoid this problem by using a Residue Number System (RNS) where large numbers are represented as a set of smaller, independent residues via the Chinese Remainder Theorem (CRT). RNS variants of BGV, BFV and CKKS are the standard in practice; OpenFHE, for example, only supports the RNS versions of these schemes [BAB+22].

Definition 2.13. Canonical Form We are interested in polynomials in the ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$ with degree (up to) $N - 1$ and coefficients in $[-Q/2, Q/2)$, and different representations of this polynomial. The canonical form (2.11) of such a polynomial a is the form most is familiar with.

$$a = c_0 + c_1X + \cdots + c_{N-1}X^{N-1} = \sum_{i=0}^{N-1} c_i X^i \quad (2.11)$$

Definition 2.14. Coefficient Form As polynomials have a well-known/predictable structure, we can omit the redundant X variable and represent $a \in R_Q$ uniquely by its coefficients. In this thesis I will consider these forms functionally equivalent, and refer to both (2.11) and (2.12) as *canonical*.

$$a = (c_0, c_1, \dots, c_{N-1}) \quad (2.12)$$

Let $Q = q_0 \dots q_{k-1}$ with $\gcd(q_i, q_j) = 1 \ \forall i \neq j$, so that CRT allows decomposition into smaller residues. If $\max_i \{q_i\}$ fits in a computer word then we can use CRT on the coefficients of a polynomial to work exclusively with integers that fit inside a computer word.

Definition 2.15. First RNS Form (CRT Form) Given a polynomial $a \in R_Q$ where $Q = q_0 q_1 \dots q_{k-1}$ is chosen as a product of k small primes (small meaning they each fit inside a computer register), each coefficient c_i is uniquely represented by its k residues via the Chinese Remainder Theorem. Then the **CRT form** of a (2.13) is a collection of smaller polynomials called *towers*, *residues* or *limbs*.

$$\begin{pmatrix} a \bmod q_0 \\ a \bmod q_1 \\ \vdots \\ a \bmod q_{k-1} \end{pmatrix} = \begin{pmatrix} c_0 \bmod q_0 & c_1 \bmod q_0 & \dots & c_{N-1} \bmod q_0 \\ c_0 \bmod q_1 & c_1 \bmod q_1 & \dots & c_{N-1} \bmod q_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod q_{k-1} & c_1 \bmod q_{k-1} & \dots & c_{N-1} \bmod q_{k-1} \end{pmatrix} \quad (2.13)$$

Addition and subtraction of two polynomials in R_Q is equivalent to element-wise addition/subtraction of their CRT forms (modulo q_i). As each element fits within a computer word, no carries propagate between them, and all pairs of elements can be added or subtracted in parallel.

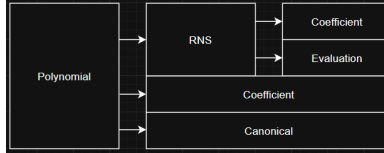
2.2.1 Number Theoretic Transform

The Number Theoretic Transform (NTT) converts a negacyclic convolution in the coefficient domain into point-wise multiplication in the NTT domain, and multiplication in R_{q_i} is exactly such a convolution of the coefficient vectors. Applying the NTT to each residue of the CRT form (2.13) therefore makes polynomial multiplication element-wise as well. The negacyclic NTT of size N requires each modulus to satisfy $q_i \equiv 1 \pmod{2N}$, which constrains the choice of RNS primes.

Definition 2.16. Second RNS form (Double CRT/NTT/Evaluation) By applying the NTT to the CRT form (2.13) we gain the ability to perform polynomial multiplication in parallel, without losing the ability to perform addition/subtraction in parallel.

$$\begin{pmatrix} \text{NTT}(a \bmod q_0) \\ \text{NTT}(a \bmod q_1) \\ \vdots \\ \text{NTT}(a \bmod q_{k-1}) \end{pmatrix} = \begin{pmatrix} \hat{a}_{0,0} & \hat{a}_{0,1} & \dots & \hat{a}_{0,N-1} \\ \hat{a}_{1,0} & \hat{a}_{1,1} & \dots & \hat{a}_{1,N-1} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{a}_{k-1,0} & \hat{a}_{k-1,1} & \dots & \hat{a}_{k-1,N-1} \end{pmatrix} \quad (2.14)$$

Remark 2.17. Rather confusingly, OpenFHE calls this form *Coefficient* format. Both RNS forms are represented by an `DCRTPoly` object, so a polynomial in CRT form is represented by a `DCRTPoly` object in `Format::Coefficient` mode.



The evaluation form (2.14) is the preferred form, as it supports parallel addition, subtraction and multiplication, but it still does not natively support *all* necessary operations. Because switching between canonical/vector, CRT and NTT mode is relatively expensive — requiring (inverse) CRTs and NTTs — much of the relevant literature [GHS12; BEHZ16; HPS18; KPZ21] improves upon existing functions by finding new approaches and approximations that limit the number of conversions required.

2.2.2 Base Conversion

Definition 2.18. RNS Basis. Given a large modulus $Q = \prod_{i=0}^{k-1} q_i$ composed of pairwise coprime factors, we define the *RNS basis* $\mathcal{B}_Q = \{q_0, q_1, \dots, q_{k-1}\}$ as the set of RNS moduli used in the RNS/CRT representation (2.13).

$$\mathbb{Z}_Q \cong \mathbb{Z}_{q_0} \times \mathbb{Z}_{q_1} \times \dots \times \mathbb{Z}_{q_{k-1}} \quad (2.15)$$

Definition 2.19. Base conversion Let $\mathcal{B}_Q = \{q_0, \dots, q_{k-1}\}$ and $\mathcal{B} = \{b_0, \dots, b_{\ell-1}\}$ be RNS bases with $\gcd(Q, B) = 1$, where $B = \prod_j b_j$. Base conversion is the map that takes the representation of x in $\mathcal{B}_Q$ to the representation of $[x]_Q$ in $\mathcal{B}$, i.e.

$$([x]_{q_0}, \dots, [x]_{q_{k-1}}) \mapsto ([x]_{b_0}, \dots, [x]_{b_{\ell-1}})$$

When the residues in $\mathcal{B}$ are appended to those in $\mathcal{B}_Q$, so that x becomes represented in the *extended* basis $\mathcal{B}_Q \cup \mathcal{B}$ for the modulus QB , the operation is also referred to as **base extension**.

Remark 2.20. *Extend mod notation for RNS polynomials:* $[a]_{\mathcal{B}_Q} = ([a]_{q_0}, \dots, [a]_{q_{k-1}})$ for the basis $\mathcal{B}_Q = \{q_i\}_{i=0}^{k-1}$ means a must be in RNS form

The naive approach to base conversion reconstructs a from its residues through inverse CRT, and then reduces the result modulo each modulus of the new basis. This reintroduces the multi-precision arithmetic that RNS was meant to avoid. Cheaper but approximate alternatives exists, described next.

Fast Base Conversion

Introduced by [BEHZ16], fast base conversion performs operates directly on the residues (in CRT mode), avoiding the CRT reconstruction and multi-precision integers entirely. *(Does not work in evaluation mode)*. The price is approximation, since (2.16) produces not x but $x + \alpha_x Q$ for some integer $\alpha_x \in [0, k)$. *Computing the exact overflow α_x is possible but would require costly operations in RNS, and is unnecessary in extension by P . Introduce the notation $\hat{q}_i = Q/q_i$ here?*

$$\text{FastBConv}(x, \mathcal{B}_Q, \mathcal{B}) = \left(\left[\sum_{i=0}^{k-1} \left[x_i \left(\frac{Q}{q_i} \right)^{-1} \right]_{q_i} \times \frac{Q}{q_i} \right]_b \right)_{b \in \mathcal{B}} \quad (2.16)$$

Fast Base Extension

We can use (2.16) as a subroutine in base extension. Only the new residues $\mathcal{B}_Q \mapsto \mathcal{B}_P$ need to be calculated, and since $\mathcal{B}_{QP} = \mathcal{B}_Q \cup \mathcal{B}_P$ the original residues are kept unchanged.

Algorithm 2.1 Fast Base Extension

Input: $[a]_{\mathcal{B}_Q}^{\text{NTT}}$ and a basis $\mathcal{B}_P$ **Output:** $[a + \epsilon]_{\mathcal{B}_{QP}}^{\text{NTT}}$ where $\epsilon = \alpha_a Q$ for some $\alpha_a \in [0, k)$

- 1: $a_Q := \text{InverseNTT}(a)$ ▷ Convert to CRT form
 - 2: $a_P := \text{FastBConv}(a_Q, \mathcal{B}_Q, \mathcal{B}_P)$
 - 3: **return** $a \cup \text{NTT}(a_P)$
-

Remark 2.21. The factors $\hat{q}_i^{-1} = [(\frac{Q}{q_i})^{-1}]_{q_i}$ and $\hat{q}_i = \frac{Q}{q_i}$ can be pre-calculated and stored in memory (discussed in section 3.1).

Scale

Note that the error $\alpha_x Q \pmod{QP}$ produced by Algorithm 2.1 would be completely removed if we multiplied by P . This observation leads to the specialized operation “*extend and scale by P*”. I refer to this operation as $\text{SCALE}(x, P) \mapsto [xP]_{\mathcal{B}_{QP}}$.

$$\begin{aligned}
 x' &= x + \alpha_x Q \pmod{QP} \\
 x'P &= xP + \alpha_x QP = xP \pmod{QP} \\
 \text{SCALE}(x, P) &= \text{FASTBASEEXTEND}(x, \mathcal{B}_Q, \mathcal{B}_P) \cdot P
 \end{aligned} \tag{2.17}$$

Furthermore, in RNS, the P -towers $\pmod{q_i \forall i}$ all become 0 when multiplied by $P = p_0 \dots p_{k'-1}$. This operation is therefore far cheaper in reality as it can be executed by only appending k' empty towers with the p_i moduli to x , and multiplying each Q -tower by P .

Algorithm 2.2 SCALE(x, P)

Input: $[x]_{\mathcal{B}_Q}^{\text{NTT}}$ with $\|\mathcal{B}_Q\| = k_q$ and a basis $\mathcal{B}_P$ with $\|\mathcal{B}_P\| = k_p$ **Output:** $[xP]_{\mathcal{B}_{QP}}^{\text{NTT}}$

- 1: **for** $(x_i, q_i) \in x$ **do**
 - 2: $x_i \leftarrow x_i \cdot P \pmod{q_i}$
 - 3: **end for**
 - 4: **for** $i \in [0, k_p)$ **do**
 - 5: $x_{k+i} \leftarrow 0 \pmod{\mathcal{B}_P[i]}$ ▷ Append empty p_i -tower to x
 - 6: **end for** **return** x
-

Modulus Switching

In RNS, modulus switching — also referred to as *mod down* — from Q to $Q' = Q/q_{k-1}$ amounts to *dropping the last limb* with a small correction, computed limb-wise without CRT reconstruction:

$$[c']_{q_i} = [q_{k-1}^{-1} (c - \delta)]_{q_i}, \quad 0 \leq i < k-1, \tag{2.18}$$

where $\delta \equiv c \pmod{q_{k-1}}$ and, for BGV, $\delta \equiv 0 \pmod{t}$ so that decryption correctness is maintained. The noise shrinks by a factor q_{k-1} , matching the classical modulus switch.

Approximate Mod Down

Modulus switching exactly requires multiprecision integers.. can use FBE again... exploits the fact that the polynomial is pre-multiplied by P ...

Uses $\delta = \text{FASTBConv}([x]_{\mathcal{B}_P}, \mathcal{B}_P, \mathcal{B}_Q)$

$$\text{APPROXMODDOWN}(x) = [P^{-1} (x - \text{FASTBConv}([x]_{\mathcal{B}_P}, \mathcal{B}_P, \mathcal{B}_Q))]_{\mathcal{B}_Q} \quad (2.19)$$

For BGV the converted term must additionally be corrected modulo t to preserve the plaintext [KPZ21].

2.3 Key-Switching

- A problem that arises in several areas of FHE is that of multiplying a ciphertext by some large number while keeping the error low.
- The techniques covered in this section typically occur in the context of keyswitching, but we remove them entirely from that context and consider only the techniques themselves.
- The gadget decomposition is not restricted to managing the noise in the scalar multiplication of ciphertexts, it is also central in the design of most FHE schemes as an auxiliary tool for certain FHE procedures; e.g., [BGV14, MP12, Bra12, GSW13, AP14, DM15, GINX16, CKKS17, CGGI20, BIP+22] [CKKS; Bra12; GSW13; DM14; CGGI18]

$$(a \cdot s + \Delta m + \epsilon) \cdot X = aX \cdot s + \Delta X m + X\epsilon \quad (2.20)$$

2.3.1 Brakerski-Vaikuntanathan

Brakerski and Vaikuntanathan [BV11] first proposed decomposing ciphertext elements into small digits as a means of controlling the noise growth of this operation. The idea has since been generalized into the idea of *gadget decomposition* and applied in a variety of forms, including digit and CRT (RNS) decomposition. Following the convention of [KPZ21; BAB+22], we refer to this family of decomposition-based techniques collectively as *BV*.

Definition 2.22. Digit Decomposition Given an integer $\gamma \in \mathbb{Z}_q$ for a modulus $q \geq 2$, base β and decomposition parameter ℓ , the digit decomposition $\text{Decomp}(\gamma) = (\gamma_0, \gamma_1, \dots, \gamma_{\ell-1}) \in \mathbb{Z}_\beta^\ell$ outputs an ordered list of ℓ base- β digits satisfying

$$\gamma = \gamma_0 \frac{q}{\beta} + \gamma_1 \frac{q}{\beta^2} + \dots \gamma_{\ell-1} \frac{q}{\beta^\ell} = \sum_{i=0}^{\ell-1} \gamma_i \frac{q}{\beta^{i+1}}$$

Digit decomposition has two important properties. First, if the digits γ_i of $\text{Decomp}(\gamma)$ are small, then $\langle \text{Decomp}(\gamma), \epsilon \rangle \leq \ell \cdot \max\{\gamma_i \cdot \epsilon_i\} \leq \beta\epsilon \ll \gamma\epsilon$. This is the property that allows some control over the noise growth. Second, two integers can be multiplied through the decomposition. By construction, $\langle \text{Decomp}(\gamma), g \rangle = \gamma$ for the fixed vector $g = (q/\beta, \dots, q/\beta^\ell)$, meaning $\langle \text{Decomp}(\gamma), g \cdot x \rangle = \gamma \cdot x$ for an arbitrary integer $x \in \mathbb{Z}$. For either of these properties, the particular form of g is unimportant as long as the digits γ_i remain small. Furthermore, as we are generally working with some intrinsic error in FHE, [CGGI18] proposed allowing some error ϵ in the reconstruction, giving the definition from their paper verbatim:

Definition 2.23. (Abstract) Gadget Decomposition An efficient algorithm $\text{Decomp}^{g, \beta, \epsilon}(v)$ is a valid decomposition on the gadget $g \in \mathbb{Z}_q^\ell$ with quality $\beta \in \mathbb{Z}^+$

and precision $\epsilon \in \mathbb{Z}^+$ if and only if for any RLWE sample $v \in R_Q$ it efficiently and publicly outputs a small vector $u \in \mathbb{Z}_q$ such that $\|u\|_\infty \leq \beta$ and $\langle u, g \rangle \leq v + \epsilon$.

- Other (equivalent) form more convenient to work with, where the output of $\text{Decomp}^{g,\beta,\epsilon}(v) \rightarrow D(v)$ and $g \cdot u \rightarrow P(u)$ are considered.

Definition 2.24. Gadget Property Two vectors $P(x) \in \mathbb{Z}_Q^\ell$ and $D(y) \in \mathbb{Z}_\beta^\ell$ are said to have the *gadget property* if and only if they satisfy (2.21) for any pair of RLWE samples $(x, y) \in R_Q^2$ with precision $\epsilon \in \mathbb{Z}_q^+$ and quality β where $\|D(y)\|_\infty \leq \beta$. If $\epsilon = 0$ then $P(x)$ and $D(y)$ are said to have the *exact* gadget property.

$$\langle P(x), D(y) \rangle = x \cdot y + \epsilon \pmod{Q} \quad (2.21)$$

Lemma 2.25. *The composition of two gadget decompositions is itself a valid gadget decomposition.*

$$\begin{aligned} \langle P_1(u), D_1(v) \rangle &= u \cdot v + \epsilon_1, & \langle P_2(u), D_2(v) \rangle &= u \cdot v + \epsilon_2 \pmod{Q} \\ P_1 \circ P_2 = P_1(P_2(u)) &\implies \langle P_1 \circ P_2, D_1 \circ D_2 \rangle &= u \cdot v + 2\epsilon \pmod{Q} \end{aligned}$$

Proof. This is clear from applying the gadget property twice.

$$\langle P_1(P_2(u)), D_1(D_2(u)) \rangle = P_2(u) \cdot D_2(u) + \epsilon_1 = u \cdot v + \epsilon_2 + \epsilon_1 \pmod{Q}$$

□

RNS Instantiation

Digit decomposition cannot be applied directly the RLWE sample in an RNS scheme, where the system only has access to the residues. When Bajard et. al. first implemented the BV cryptosystem using RNS [BEHZ16] they exchanged digit decomposition for CRT decomposition, exploiting the fact that this decomposition is already required by the RNS.

Definition 2.26. Chinese Remainder Theorem is clearly a gadget

$$CRT : \sum N \quad (2.22)$$

- CRT idempotent used in [BEHZ16]
- HPS optimization for specifically keyswitching [HPS18]
- Definition: gadget with $\beta = Q/2, \epsilon = 0$

$$P_{\text{HPS}}(u) = () \quad (2.23)$$

$$D_{\text{HPS}}(v) = () \quad (2.24)$$

- Note on HPS gadget = RNS limb lookup $\rightarrow$ section 3.1
- HPS is exact but noise is large ($\beta = Q/2$, $\epsilon = 0$); can apply digit decomposition to each residue polynomial as per Theorem 2.25, lowering $\beta \rightarrow B/2$ and still exact ($\epsilon = 0$) if $B = \lfloor \max q_i / \ell \rfloor + 1$

2.3.2 RGSW

- GSW introduced in [GSW13]
- and its ring variant [DM14]
- two-dimensional gadget property

Definition 2.27. RGSW Ciphertext Given a gadget vector g

$$C = Z + \mu \cdot G \quad (2.25)$$

- We want a product between RLWE and RGSW ciphertexts
- The RLWE-RGSW relationship is similar to vector vs matrix

Definition 2.28. External Product

$$\boxtimes : \text{RLWE}(\mu) \boxtimes \text{RGSW}(\gamma) = \text{RLWE}(\mu) \quad (2.26)$$

$$x \boxtimes C = G^{-1}(x) \cdot C \quad (2.27)$$

Lemma 2.29. Gadget Matrix Construction *Two vectors $P(u), D(v)$ that satisfy the gadget property (2.21) define a valid pair of gadget matrices with $g = P(1)$:*

$$G = I_2 \otimes P(1) \quad (2.28)$$

$$G^{-1}(x) = [D(x_0) | D(x_1)] \quad (2.29)$$

$$s.t. \ G^{-1}(x) \cdot G = x + \epsilon \pmod{Q} \quad (2.30)$$

Proof.

$$G^{-1}(x) \cdot G = [D(x_0) | D(x_1)] \begin{pmatrix} P(1) & 0 \\ 0 & P(1) \end{pmatrix}$$

$$(D(x_0) \cdot P(1), D(x_1) \cdot P(1)) = (x_0 + \epsilon_0, x_1 + \epsilon_1) = x + \epsilon \pmod{Q}$$

□

Remark 2.30. The external product has two sources of noise, as detailed in Theorem 2.30. The precision ϵ of the gadget decomposition is scaled by μ . This is why μ is typically constrained to $\{0, 1\}$ when an inexact gadget is used. The other source is the term $G^{-1}(x) \cdot Z$ which scales the sum of the masking errors by the

gadgets quality: $\implies \epsilon \leq \beta \sum_i^{2\ell-1} e_i \leq \beta \max_i \|e_i\|$. Consequently, the error of x being decomposed is always $\leq \beta$ but if the errors in Z are large, ... thats not good.

$$\begin{aligned} x \cdot C &= G^{-1}(x)(Z + \mu G) \\ G^{-1}(x) \cdot Z + \mu(G^{-1}(x) \cdot G) \\ G^{-1}(x)Z + \mu(x + \epsilon_g) \end{aligned}$$

where $\|G^{-1}\|_\infty \leq \beta$

$$\text{Using (2.29): } G^{-1}(x)Z = [D(x_0)|D(x_1)] \begin{pmatrix} \text{RLWE}(0) \\ \vdots \\ \text{RLWE}(0) \end{pmatrix}$$

$$\text{Let } D_i = G^{-1}(x)[i] \text{ and } Z_i = (a_i \cdot s + \epsilon_i, a_i)$$

$$\begin{aligned} \implies G^{-1}(x) \cdot Z &= \left(\sum_{i=0}^{2\ell-1} a_i D_i s + \epsilon_i D_i, \sum_{i=0}^{2\ell-1} a_i D_i \right) \\ &= \left(\left[\sum_{i=0}^{2\ell-1} a_i D_i \right] s + \sum_{i=0}^{2\ell-1} \epsilon_i D_i, \sum_{i=0}^{2\ell-1} a_i D_i \right) \end{aligned}$$

$$\text{Let } a = \sum_{i=0}^{2\ell-1} a_i D_i \text{ and } \epsilon = \sum_{i=0}^{2\ell-1} \epsilon_i, \text{ and recall } \|D(x_i)\|_\infty \leq \beta$$

$$\begin{aligned} (a \cdot s + \sum_{i=0}^{2\ell-1} \epsilon_i D_i, a) &\leq (a \cdot s + \sum_{i=0}^{2\ell-1} \epsilon_i \beta, a) \\ \therefore G^{-1}(x) \cdot Z &\leq (a \cdot s + \beta \epsilon, a) \end{aligned}$$

So clearly, the error sources are:

- $\mu \epsilon_g$ which is 0 if the gadget is exact
- $\beta \sum e_z$ the sum of

- We also need a product definition between two (Ring) Gentry-Sahai-Waters (RGSW) ciphertexts
- Using the fact that an RGSW ciphertext is a matrix of RLWE ciphertexts we can repeat the external product on each row

Definition 2.31. Internal Product

$$\boxtimes : \text{RGSW}(a) \boxtimes \text{RGSW}(b) = \text{RGSW}(a \cdot b) \quad (2.31)$$

$$A \boxtimes C = \begin{bmatrix} A_0 \boxtimes C \\ A_1 \boxtimes C \\ \vdots \\ A_{2^\ell-1} \boxtimes C \end{bmatrix} \quad (2.32)$$

Remark 2.32. Since the quality β of gadget decomposition is determined only by the

- **Future work/results:** Though digit decomposition is often used, emphasize the external product only requires $G^{-1}(x) \cdot G \approx x \bmod Q$

2.4 (Somewhat) Practical Anonymous Router

Chapter 3

Methodology

- Gap: TFHE-style RGSW products (external and internal) were not available in BGV-RNS; a C++ library (**core**) was developed to add this functionality on top of OpenFHE [BAB+22].
- Second artifact: an instantiation of the sPAR protocol [DP25] (**server**, **client**) built on **core**.
- The two are independent: **core** is usable on its own, and the sPAR implementation depends only on the abstract interface of section 3.1, not on a particular instantiation.
- Chapter structure: section 3.1 documents the library architecture and its canonical (BV) instantiation; section 3.2 the sPAR instantiation; section 3.3 the experimental methodology. All measurements are deferred to chapter 4.

3.1 Implementation Details of the Core Library

3.1.1 Architecture

- **core** (directory `libs/core` of [Ang26]) extends OpenFHE’s `CryptoContext` with an abstract interface `IExtendedContext` exposing the RGSW operations of Table 3.1.
- Interface/implementation separation: any number of concrete instantiations (one per gadget construction) implement the same interface; consuming code depends only on the interface, not on the instantiation.
- The interface mirrors OpenFHE’s API and naming conventions (**Encrypt***, **Eval***); a user familiar with OpenFHE can adopt the library directly.
- Codebase layout (`libs` directory): **core** (RGSW functionality), **server** and **client** (sPAR server/client functions built on **core**). Auxiliary directories: **benchmark** and **test** (see section 3.3), and **scripts** (Python; security/parameter estimation via the lattice estimator).

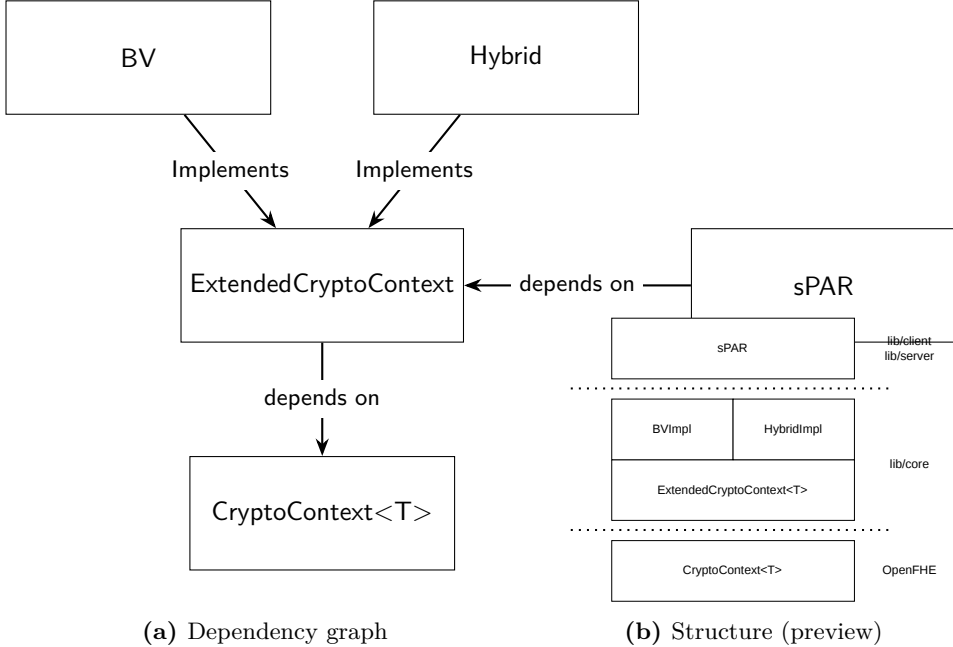


Table 3.1: Operations provided by `IExtendedContext`

<code>ENCRYPTRGSW(pk, pt)</code>	RGSW encryption of a plaintext
<code>MAKEPUBLICRGSW(pk, pt)</code>	Noiseless (public) RGSW, $Z = 0$
<code>EVALEXTERNALPRODUCT(rlwe, rgsw)</code>	External product $\boxtimes$
<code>EVALINTERNALPRODUCT(lhs, rhs)</code>	Internal product $\boxtimes$
<code>EVALADDRGSW(lhs, rhs)</code>	Row-wise RGSW addition
<code>EVALSUBRGSW(lhs, rhs)</code>	Row-wise RGSW subtraction
<code>EVALMULRGSW(rgsw, pt)</code>	RGSW–plaintext multiplication

- RGSW data structure: a vector of RLWE ciphertexts — the rows of $C = Z + \mu G$ (2.25) — each an ordinary OpenFHE ciphertext. The number of rows is determined by the gadget ($2k\ell$ for the BV instantiation below).
- `MAKEPUBLICRGSW` outputs μG exactly ($Z = 0$, i.e. $a = 0$, $\epsilon = 0$): not a ciphertext in the cryptographic sense but a plaintext carried in RGSW form, enabling products with true ciphertexts. Used for server-side constants in section 3.2.
- Two concrete implementations are bundled. This chapter documents the canonical one, built on the layered RNS/digit gadget of section 2.3 and referred to as the *BV* instantiation; it is canonical because it matches how the external product is defined in the literature (via digit decomposition) [CGGI18; DP25] and uses true RGSW ciphertexts. The second implementation, based on modulus extension, is deferred to chapter 4.

Remark 3.1. The implementations are coupled to OpenFHE’s `CryptoContextBGVRNS`. Since OpenFHE exposes a unified interface for BGV and BFV, and the RGSW products are scheme-agnostic, extending the library to BFV would require little work.

3.1.2 The BV Instantiation

The Gadget

- Instantiates the layered gadget of Theorem 2.25: the CRT/RNS gadget P_{HPS} on the limbs, composed with signed base- B digit decomposition of each limb.
- Concrete gadget pair, indexed by $(i, j) \in [0, \ell) \times [0, k)$; let $\tilde{e}_j \in \mathbb{Z}_Q$ denote the CRT idempotent with $\tilde{e}_j \equiv 1 \pmod{q_j}$, $\tilde{e}_j \equiv 0 \pmod{q_{j'}}$ for $j' \neq j$:

$$P_{\text{BV}}(v)_{(i,j)} = B^i \tilde{e}_j v, \quad D_{\text{BV}}(u)_{(i,j)} = d_i([u]_{q_j}),$$

where $d_i(\cdot)$ is the i -th signed base- B digit and the (small) digit is embedded into R_Q (“broadcast” to every tower).

- Exactness and quality: since $\sum_i d_i([u]_{q_j}) B^i = [u]_{q_j}$,

$$\langle D_{\text{BV}}(u), P_{\text{BV}}(v) \rangle = \sum_j \tilde{e}_j [u]_{q_j} v = u \cdot v \pmod{Q},$$

i.e. the gadget is exact ($\epsilon = 0$) with quality $\beta = B/2$.

- In RNS storage $\tilde{e}_j$ zeroes every tower except tower j , so $P_{\text{BV}}(v)_{(i,j)}$ occupies a single native tower; the zero towers are never touched (neither stored computations nor products), cf. section 3.1.2.
- Parameter selection: the context takes only ℓ and derives the digit width

$$b = \log_2 B = \left\lfloor \max_j \lceil \log_2 q_j \rceil / \ell \right\rfloor + 1,$$

the smallest power-of-two base covering the largest limb with ℓ digits and one extra bit of headroom; the headroom absorbs the decomposition offset of section 3.1.2. This keeps the gadget exact regardless of how OpenFHE sizes the individual q_j (the first prime is generated with a different bit size than the rest).

- B a power of two enables branchless digit extraction via bitwise operations (Table 3.2).

RGSW Encryption

- By (2.25), $C = Z + \mu G$ with $G = I_2 \otimes P_{\text{BV}}(1)$; the payload added to row (i, j) is $\mu \cdot B^i \tilde{e}_j = [\mu B^i]_{q_j} \tilde{e}_j$.
- Structure: in RNS storage row (i, j) holds the single native polynomial $[\mu B^i]_{q_j}$ in tower j ; all other towers of the payload are zero. Collecting the nonzero towers across rows reassembles the RNS representations of $\mu, \mu B, \dots, \mu B^{\ell-1}$:

$$P_{\text{BV}}(\mu) \cong \begin{bmatrix} [\mu]_{q_0} & \cdots & [\mu B^{\ell-1}]_{q_0} \\ \vdots & \ddots & \vdots \\ [\mu]_{q_{k-1}} & \cdots & [\mu B^{\ell-1}]_{q_{k-1}} \end{bmatrix} = ([\mu]_{\mathcal{B}_Q}, \dots, [\mu B^{\ell-1}]_{\mathcal{B}_Q})$$

- Consequently no decomposition is performed at encryption time: row (i, j) requires one native tower product $[\mu]_{q_j} \cdot [B^i]_{q_j}$, computed directly in the NTT domain and added into tower j of component 0 (rows $0, \dots, k\ell - 1$) resp. component 1 (rows $k\ell, \dots, 2k\ell - 1$) of a fresh encryption of zero.
- The factors $[B^i]_{q_j}$ are precomputed once per context ($k\ell$ scalars).
- The $k\ell$ row pairs are independent; Algorithm 3.1 parallelizes over them.
- MAKEPUBLICRGSW is identical except $Z = 0$: the rows are zero polynomials carrying only the payload tower (and the encryption metadata).

External Product

- Computes $x \boxdot C = G^{-1}(x) \cdot C$ per ?? with the gadget above; the sum runs over $2k\ell$ rows.
- Signed vs. unsigned digits: unsigned extraction ($d_i \in [0, B)$) is trivially parallel but has quality $\beta = B$; centering ($d_i \in [-B/2, B/2)$) halves the quality to

Algorithm 3.1 Parallel RGSW-BV Encryption**Input:** Decomposition parameter ℓ , base B **Input:** RNS polynomial $[\mu]_{B_Q}^{\text{NTT}} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$ **Output:** RGSW(μ)

- 1: Initialize empty array C with $2k\ell$ slots
- 2: **for** $r \leftarrow [0, k\ell)$ **in parallel do**
- 3: $(i, j) \leftarrow (r \bmod \ell, \lfloor r/\ell \rfloor)$ $\triangleright i$ -th power B^i , j -th tower
- 4: $mg \leftarrow \text{TOWERMUL}(\mu, j, B^i)$
- 5: $C_r \leftarrow \text{ENCRYPT}(0) + (mg, 0)$
- 6: $C_{r+k\ell} \leftarrow \text{ENCRYPT}(0) + (0, mg)$
- 7: **end for**
- 8: **return** C

$\beta = B/2$ at the cost of carry propagation between digits, which serializes the extraction.

- Offset trick — carry-free signed extraction. Precompute the global constant

$$\text{offset} = \frac{B}{2} \sum_{i=0}^{\ell-2} B^i = \frac{B}{2} \cdot \frac{B^{\ell-1} - 1}{B - 1}.$$

For $x \in [0, q_j)$ set $u_i = \lfloor (x + \text{offset})/B^i \rfloor \bmod B$ and

$$d_i = u_i - \frac{B}{2} \quad (i < \ell - 1), \quad d_{\ell-1} = u_{\ell-1}.$$

Then $\sum_i d_i B^i = x$ exactly, $d_i \in [-B/2, B/2)$ for $i < \ell - 1$, and the top digit is non-negative and bounded by $B/2$ (the headroom bit). Each u_i depends only on x , so all $k\ell$ digits of all N coefficients extract independently (branchless: one shift and one mask each, Table 3.2).

- Negative digits are stored per tower as $q_t - |d_i|$; centering per Algorithm 3.2.
- Digit extraction requires the coefficient representation; the NTT round trips are the dominant cost of the BV-RNS gadget. The implementation is staged so every transform is an independent single-tower task:
 1. inverse NTT of each tower of (c_0, c_1) — $2k$ tasks;
 2. digit extraction, signed lift, and forward NTT per (digit slot, target tower) — $2k\ell \cdot k$ tasks;
 3. accumulation $\sum_s d_s \cdot C_s$ per (component, tower) — $2k$ contention-free tasks (no reduction across threads).
- Cost per product: $2k$ inverse and $2k^2\ell$ forward single-tower NTTs, and $4k^2\ell$ native-polynomial multiply-accumulates.

Bitwise Operations The library takes $b = \log_2 B$ as the parameter and derives $B = 2^b$, so that the arithmetic of digit extraction reduces to constant-time bitwise

operations (Table 3.2); in particular, reduction modulo B becomes a mask and division by B a shift.

Table 3.2: Bitwise Operators

Named Function	Operator Notation	Value
LEFTSHIFT(x, b)	$x \ll b$	$x \cdot 2^b$
RIGHTSHIFT(x, b)	$x \gg b$	$\lfloor x/2^b \rfloor$
AND(x, y)	$x \wedge y$	$\sum_i x_i^{(2)} \cdot y_i^{(2)} \cdot 2^i$
MODULO(x, b)	$x \wedge [(1 \ll b) - 1]$	$x \bmod 2^b$

Algorithm 3.2 CENTER(x, b)

Input: $x, b \in \mathbb{Z}$

Output: $[x]_{2^b} \in [-2^{b-1}, 2^{b-1})$

- 1: mask $\leftarrow$ LEFTSHIFT($1, b$) $- 1$
 - 2: half $\leftarrow$ LEFTSHIFT($1, b - 1$)
 - 3: **return** AND($x + \text{half}, \text{mask}$) $- \text{half}$
-

Internal Product

- Implemented as $2k\ell$ external products, one per row of the left operand, per ??.
- Ordering matters: in $X \boxtimes Y$ the rows of X are decomposed, so the noise of X enters the result scaled only by the (binary) message of Y , while the noise of Y enters scaled by β and summed over rows (Theorem 2.32). In a chain of products the accumulating (noisy) operand must therefore be the left operand; the fresh, low-noise operand the right.
- This rule dictates the order of products in SERVER::WRITE (section 3.2); its quantitative effect (30–40 bits) is reported in chapter 4.

SIMD Batching

- With batching-compatible parameters, packed encoding (MakePackedPlaintext in place of MakeCoefPackedPlaintext) turns every RGSW encryption and external/internal product into up to N independent slot-wise products, at no additional cost relative to the same parameters without batching.

3.2 Implementation Details of sPAR

3.2.1 Deviations from the Specification

- Implemented: setup, client encryption, the server write step, and (multiparty) decryption. Omitted: bucket sorting and the protocol extras (equivocation protection, etc.).
- Sorting is computationally negligible relative to the write step — estimated < 100 ms and highly parallelizable. **FLAG: missing citation for the estimate.**
- Indices are submitted as D one-hot RGSW indicator vectors $z_d \in \text{RGSW}(\{0, 1\})^\eta$, $d \in [0, D)$, rather than as $\log \eta$ encrypted bits per index.
- The write step is split in two: pre-processing and the write loop itself. **FLAG: state precisely what the pre-processing computes.**

3.2.2 Procedural Design

- The libraries implement the protocol *functions*; they do not provide state-maintaining server or client processes. The tests and benchmarks compose these functions into the full protocol.
- Rationale: decouples the functionality from the benchmarking harness without implementing real communication (serialization, error handling, coordination of multiple machines), which is orthogonal to the questions studied here.

3.2.3 Server Write Step

- The central algorithm: performs the oblivious writes and all homomorphic multiplications; everything else in the round is encryption or decryption.
- State: $I \in \text{RGSW}(\{0, 1\})^{\eta \times K}$ (slot-availability indicators, initially 1) and $L \in \text{RLWE}(R_t)^{\eta \times K}$ (value accumulators, initially 0).
- Noiseless initial state: the server generates the initial I , L and `notHasWritten` as public (noiseless) ciphertexts via `MAKEPUBLICRGSW` resp. zeroed RLWE elements ($a = 0$, $\epsilon = 0$). Justification: these initial values are protocol constants the server knows anyway, and the server is the adversary — nothing is hidden from it by encrypting them. Effect: setup spends no time on encryption or randomness generation; minor reduction in accumulated noise.
- The paper’s `hasWritten` flag is inverted to `notHasWritten` (initial value 1, decremented on write), which the algorithm also returns as its success indicator (1 iff no slot accepted the value).
- Order of internal products: $h = (\text{notHasWritten} \boxtimes I_{i,k}) \boxtimes z_{d,i}$, with the accumulating operand on the left per section 3.1.2; compared to the original formulation

$(z \boxtimes I) \boxtimes \text{notHasWritten}$ this improved the output noise of `SERVER::WRITE` by 30–40 bits (chapter 4).

- Deep circuits are handled with external/internal products only, as recommended by [DP25]; the value $L_{i,k} += V_r \boxtimes h$ is an external product with the RLWE-encrypted value V_r , but the indicator updates are not expressible with external products alone — internal products are required for h .

Algorithm 3.3 `Server::Write`

Input: References to the state matrices I and L

Input: RLWE-encrypted value V_r

Input: One-hot RGSW index vectors $z_d = (z_{d,0}, \dots, z_{d,\eta-1})$ for $d \in [0, D)$

Output: Updates I and L ; returns encrypted failure indicator

```

1: notHasWritten  $\leftarrow$  MAKEPUBLICRGSW(1)
2: for  $d \leftarrow [0, D)$  do
3:   for  $k \leftarrow [0, K)$  do
4:     for  $i \leftarrow [0, \eta)$  do
5:        $h \leftarrow (\text{notHasWritten} \boxtimes I_{i,k}) \boxtimes z_{d,i}$ 
6:        $L_{i,k} \leftarrow L_{i,k} + (V_r \boxtimes h)$ 
7:        $I_{i,k} \leftarrow I_{i,k} - h$ 
8:       notHasWritten  $\leftarrow$  notHasWritten  $- h$ 
9:     end for
10:   end for
11: end for
12: return notHasWritten
```

Remark 3.2. The choice $K = D = 3$ is not hard-coded: both are compile-time constants (C++ template parameters), configurable with no effect on the size of the compiled program.

3.3 Experimental Setup

3.3.1 Protocol Simulation

- A single “all-knowing” orchestrator process (with access to all key material) simulates every party and performs the setup plus one full protocol round; no networking is involved.
- Round structure, parameterized over the number of users η (**benchmark** directory):
 - each virtual user holds a unique ID and a secret key share; the joint public key is produced by OpenFHE’s chained multiparty key generation (each user extends the previous user’s public key share; the final key is the joint public key);
 - each user submits an RLWE encryption of its value and D one-hot RGSW index vectors under the joint key;
 - the server executes `SERVER::WRITE` once per user;
 - decryption is multiparty: every user produces a partial decryption; fusion yields the plaintext.
- The setup phase (key generation, state initialization) is considered free and excluded from all measurements.

3.3.2 Unit Tests and Benchmarks

- Unit tests (**test** directory) empirically validate correctness: RGSW encryption and both products are tested for the plaintexts 0, 1 and a larger message, in both coefficient-packed and SIMD variants; the full protocol is tested parameterized over η — a passing run certifies the parameter set handles η users.
- Tests check correctness at every stage and thereby localize failures (which operation failed and why), at the cost of speed.
- Benchmarks perform no correctness checks. One-shot timings suffice: client-side operations run independently per user, so the single-run time is the per-user latency, and for the server write only the per-user time is of interest.
- Tests and benchmarks share parameters and code paths, so a passing test validates exactly the computation the benchmark measures.
- The moduli q_i are randomly generated per run; empirically the results do not depend on the specific q_i , only on the bit size of Q .

3.3.3 Noise Measurement

- The testing environment always has access to the secret key, so RLWE noise is measured directly:

$$x = (c_0, c_1) = (c_1 s + \Delta m + \epsilon, c_1) \implies \epsilon = c_0 - c_1 s - \Delta m, \quad m = \text{DECRYPT}(x, s).$$

- RGSW ciphertexts are intermediate objects; two ways to measure their noise:
 1. extract the row encoding $\mu \cdot 1$ (exists whenever the gadget contains the entry 1) and measure it as an RLWE ciphertext;
 2. compute the external product with a noiseless RLWE(1) (via MAKEPUBLICRGSW-style zero-noise encryption) and measure the result.
- The latter is used: it applies to arbitrary gadgets, whereas the former assumes 1 is a gadget entry — traditional, but not guaranteed (and not true for the BV gadget rows scaled by $\tilde{e}_j$).

Chapter 4

Results and Discussion

This section begins by presenting the parameters used in benchmarking, then ...

I will close the section by discussing the results and suggest a new approach...

4.1 Parameters

I found that the parameters suggested in [DP25] results in too much noise, and are thus not capable of running the full sPAR protocol without decryption failure. Rather than tweaking these parameters until the protocol could run, I instead aim to fully utilize the RNS context by using parameters that enable SIMD batching.

Parameters were thus selected to provide $\lambda \geq 128$ bits of security at the largest possible Q while supporting SIMD batching (t prime s.t. $t \equiv 1 \pmod{N}$), then ℓ was tweaked until the sPAR protocol runs without decryption failure. The RNS primes are set slightly below 64 bits to minimize k , **with the first prime q_0 being slightly larger than the following primes by convention??** I found that this left much headroom, prompting me to lower k until $\ell = 2$ could no longer sustain the noise. The resulting parameters, listed in Table 4.1, yield a security parameter of 339.6 bits [APS15] for the underlying RLWE scheme.

Table 4.1: Base parameters for BV-RNS

Parameter	Value
decomposition (ℓ, B)	$(3, 2^{21})$
number of RNS moduli k	4
first mod size $\log q_0$	60
scaling mod size $\log q_i, i > 0$	59
ciphertext modulus order $\log Q$	2^{296}
standard deviation σ	3.19
plaintext modulus t	65537
polynomial degree N	16385
security parameter λ	339.6

- Explain difference in size between q_0 and q_i
- Explain q_i are random 59-bit primes so only their order is listed
- Data per message = $N \log(t)/8 \approx 16385 \cdot (16)/8 = 32770$ bytes ≈ 32 Kb
- sPAR data per msg = $2048 * 8/8 = 2048 = 2$ Kb
- For performance, the key is generally to minimize $k\ell$, in this case $2k\ell = 2 \cdot 4 \cdot 3 = 2 \cdot 12 = 24$ rows per RGSW!
- Note for self: if larger params is 16 times slower than expected, throughput is the same

4.2 Noise Analysis

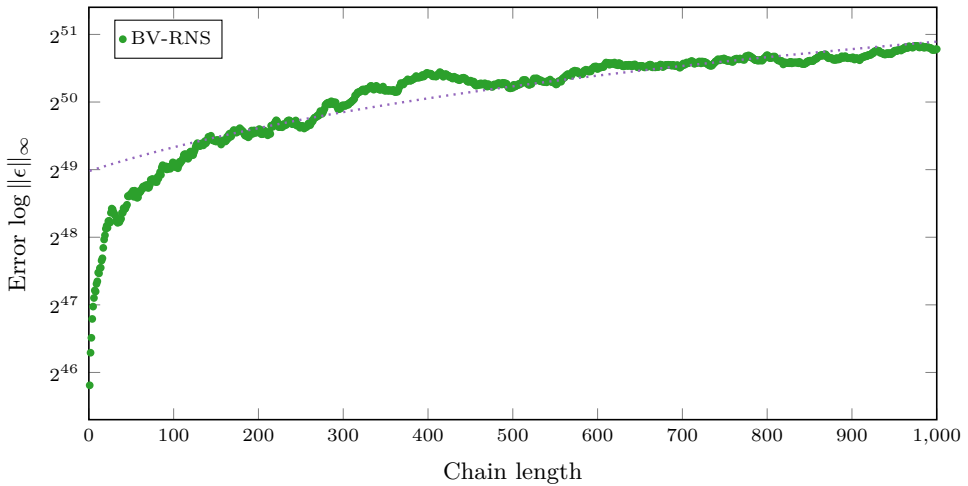
- I present the noise growth of various functions
- Using the max-norm $\|\epsilon\|_\infty = \max_i \{\epsilon_i\}$ etc.
- The absolute value does not really matter; we care more about how many bits of error
- Noise plots are therefore semi-logarithmic, with the y-axis displaying $\log(v)$

4.2.1 Core Library

External Product

- Note: LHS is the accumulated value, RHS is always fresh (due to known decomposition asymmetry that should be discussed in Theorem 2.32)
- Error is additive, less than 51 bits of noise for 1000 chained operations
- To illustrate linear growth: linear regression is plotted on the graph as a dotted line
- Use linear regression to determine max chain length: ...
- Maybe: show max chain length if RHS is accumulated noise

Figure 4.1: RNS-BV noise growth in chained external products

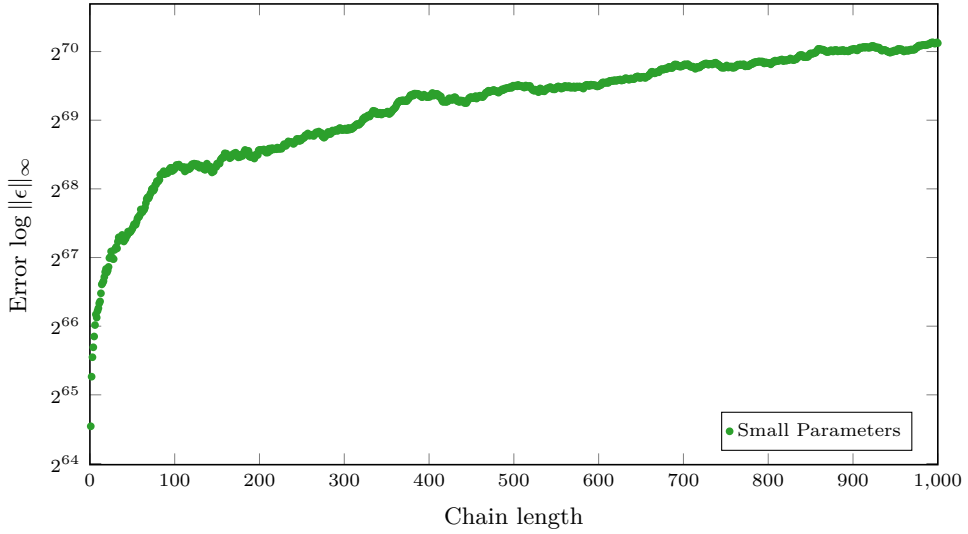


Internal product

- Note: LHS is still the accumulated value, RHS is always fresh
- Error is still additive, but each internal product is several external products
- Less than 71 bits of noise for 1000 chained operations

- This is much larger, on the order of 2^{20}

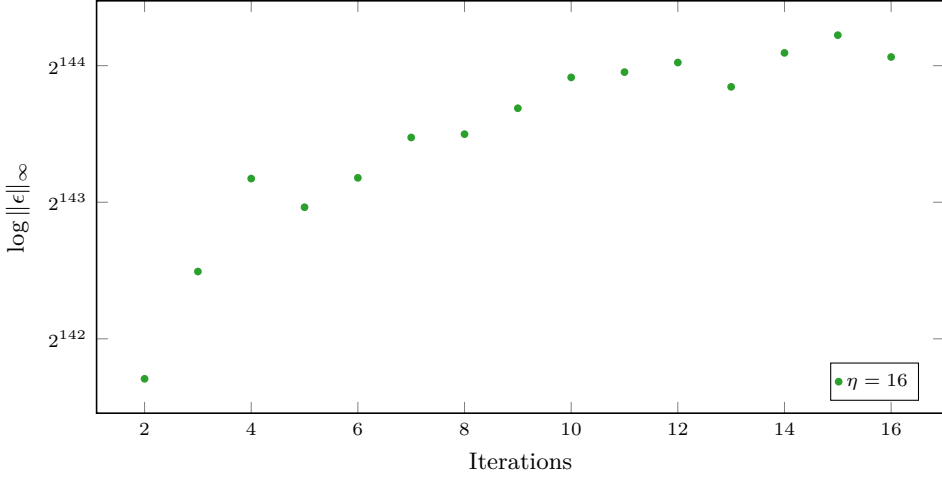
Figure 4.2: RNS-BV noise growth in chained internal products **TODO: Overlay graph of using accumulated result on rhs**



$$- 1.08139e18 \cdot x + 2.1953e20 \leq 2^{180} \implies x \leq 1.41715 \cdot 10^{36}$$

4.2.2 sPAR

- If the parameters support even the base case of $\eta = 2$ users, then the scheme can support a nearly arbitrary amount of users with the same parameters
- Noise is a big problem due to the internal product
- Getting enough noise headroom at $\lambda = 128$ requires very large parameters, making the protocol much slower than anticipated

Figure 4.3: Server::Write

- Linear regression: $1.47533e42x + 4.7166e42$
- Noise ceiling: 180 bits
- Assuming same growth: $1.47533e42\eta + 4.7166e42 \leq 2^{180} \implies \eta \leq 1.03875 \cdot 10^{12}$
- Noise growth is probably larger as η increases; have to verify

Multi-Party FHE Noise

- This section is separated from the
- My instantiation uses the default OpenFHE threshold FHE, which represents a practical (easy to set up) but not necessarily the state-of-the-art of MPC.

4.3 Performance Benchmarks

- Run on AMD Ryzen 5 5600X CPU 6-core @ 3.7 GHz with 32Gb RAM in WSL2 Ubuntu 24.04 LTS Linux

4.3.1 Core Library

Table 4.2: RGSW Operations (todo: update numbers)

Operation	Time
Encrypt RGSW	5.08 ms
External Product	1.66 ms
Internal Product	20.24 ms

- The internal product scales exponentially as it performs $2k\ell$ external products, which in turn performs $2k\ell$ multiplications = $\mathcal{O}(4k^2\ell^2)$
- Since GHS specifically only has only 2 rows, the scaling is a set factor of approx. 2 instead of $2k\ell$

4.3.2 sPAR

Table 4.3: One round of the full sPAR protocol, not adjusted to per user except Write, 6 threads

η	Total	Encrypt	Write	Partial Dec.	Server Dec.
2	8620 ms	402.53 ms	8.06798 s	140.60 ms	5.02677 ms
4	17984 ms	1652.05 ms	15.9965 s	425.13 ms	10.783 ms
8	41232 ms	6745.00 ms	33.0402 s	1472.59 ms	28.1619 ms
16	98332 ms	26382.50 ms	67.368 s	5005.41 ms	60.5488 ms

- **Discussions:** If write can use SIMD batching the write loop can be divided by up to $N \implies 4.675$ ms, 18.798 ms, 74.69 ms per round, respectively, but the other functions must also be scaled up..
- **TODO:** run single-threaded, adjust to η i.e. divide by η for encrypt, write, partial decrypt

4.3.3 Multi-Threaded Performance

- Works now, but only some functions (EncryptRGSW, BV Decompose) are optimized to take advantage of multi-threading.
- $\leq k\ell$ threads saturate, so $k = 2, \ell = 3$ has optimal performance at 6 threads.
- Ideally more threads would used to perform multiple coefficients simultaneously, but this is not implemented.

Table of improvements +X% at 1 to 6 threads

4.4 Discussions

- The results show that number of users is not a problem; noise is
- Once noise is controlled, many users can be handled due to the additive scaling of the RGSW products
- GHS improvement over BV even adjusted to same security parameter, but would require a solution to the internal product

4.4.1 Theoretical Hybrid Improvement

An alternative to gadget decomposition used in RNS key-switching is *hybrid* keyswitching [GHS12; KPZ21].

- This approach nearly fits inside the gadget decomposition framework established in section 2.3, as $\langle u \cdot P^{-1}, v \cdot P \rangle = u \cdot v \cdot P^{-1} \cdot P = u \cdot v \bmod QP$
- This actually defines a valid external product, the problem is that the P -scaling is consumed after this operation

Definition 4.1. Hybrid-RGSW Ciphertext An RGSW ciphertext that encodes μP in the extended

$$C_{QP} = Z_{QP} + \mu \begin{pmatrix} P & 0 \\ 0 & P \end{pmatrix} \pmod{QP} \quad (4.1)$$

$$\text{HYBRIDRGSW}(b) = \text{RGSW}(b \cdot P) \quad (4.2)$$

Algorithm 4.1 Hybrid-RGSW Encryption

Input: Extended basis $\mathcal{B}_P$

Input: RNS polynomial $[\mu]_{\mathcal{B}_Q}^{\text{NTT}} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$

Output: $\text{RGSW}(\mu \cdot P)$

- 1: Initialize empty array C with $2k\ell$ slots
 - 2: **for** $r \leftarrow [0, k\ell)$ **in parallel do**
 - 3: $z \leftarrow \text{HE.ENCIPHER}(0, pk_{\mathcal{B}_{QP}})$
 - 4: $mG \leftarrow P \cdot \text{BASEEXTEND}(m, Q, P)$
 - 5: $C_r \leftarrow \text{HE.ENC}(0) + (mg, 0)$
 - 6: $C_{r+\ell} \leftarrow \text{HE.ENC}(0) + (0, mg)$
 - 7: **end for**
 - 8: **return** C
-

Definition 4.2. Hybrid External Product Is valid...

Lemma 4.3. *The external product $\text{RGSW} \square \text{HYBRIDRGSW}$ is a valid product...*

Proof.

$$x_Q \square C_{QP} = P^{-1}(x_q \cdot Z_{QP} + \mu \begin{pmatrix} P & 0 \\ 0 & P \end{pmatrix}) \pmod{QP} \quad (4.3)$$

□

- This method shows far better noise handling and
- Show graphs for same parameters but $|P| \approx |Q|$ i.e. $d_{num} = 1$ and security parameter smaller but still ≥ 128 bits

Algorithm 4.2 RLWE $\boxtimes$ Hybrid-RGSW

Input: Extended basis $\mathcal{B}_P$ **Input:** RNS polynomial $[\mu]_{\mathcal{B}_Q}^{\text{NTT}} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$ **Output:** $\text{RGSW}(\mu \cdot P)$

- 1: Initialize empty array C with $2k\ell$ slots
 - 2: **for** $r \leftarrow [0, k\ell)$ **in parallel do**
 - 3: hm
 - 4: **end for**
 - 5: **return** C
-

- Problem: internal product does not work with both sides as Hybrid-RGSW and one P is consumed each external product!

Definition 4.4. Hybrid Internal Product I define the *hybrid internal product* as

$$\boxtimes_H := \text{RGSW}(a) \boxtimes_H \text{RGSW}(b \cdot P) = \text{RGSW}(a \cdot b) \quad (4.4)$$

Algorithm 4.3 Modified

Input: Extended basis $\mathcal{B}_P$ **Input:** RNS polynomial $[\mu]_{\mathcal{B}_Q}^{\text{NTT}} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$ **Output:** $\text{RGSW}(\mu \cdot P)$

- 1: Initialize empty array C with $2k\ell$ slots
 - 2: **for** $r \leftarrow [0, k\ell)$ **in parallel do**
 - 3: $h \leftarrow (I_{i,k} \boxtimes \text{nhw}) \begin{smallmatrix} \sim \\ \square \end{smallmatrix} \boxtimes_H z_{i,d}$
 - 4: **end for**
 - 5: **return** C
-

- Assuming Algorithm 3.3 is implemented using only external products, or the internal product can be implemented in the hybrid scheme, we see that the hybrid approach is generally faster and has better noise properties

Chapter 5

Conclusion and Future Work

Introduction...

5.1 Conclusion

- Achieved better time than expected with GHS
- Large noise
- Number of users

5.2 Future Work

- BGV SIMD capabilities [SV11] not utilized in write loop; `Server::Write` can perform N rounds simultaneously, but does this translate into higher throughput? A constant polynomial is the same object in SIMD batching and as a regular polynomial, so if write is changed to output binary $\{0, 1\}$ it should be possible with:
 - For each $i \in [0, N)$
 - $\text{mask} \leftarrow e_i = (0, 0, \dots, 1, \dots, 0)$ where only the i -th element is 1
 - $\text{isolated} \leftarrow h \cdot \text{mask}$
 - msg
- AVX512 not tested because time + no access to a computer that supports this instruction set; this would decrease the RNS primes to 48 bits, increasing performance and security, decreasing noise headroom
- RGSW/packed RLWE bandwidth optimization; originally planned, dropped due to time constraints, requires more complicated setup phase as RGSWs is required [CCR19; LY25]
- BFV implementation should be easy as the library sits on top of OpenFHE, it should be as easy as swapping out a few lines of code. Since BFV places its messages in the most significant bits, the decomposition gadget could be made inexact as with the powers of base, the least significant digits are dropped when ℓ is too small for B^ℓ to cover q
- The full sPAR protocols contains additional things such as equivocation protection, sorting buckets etc.

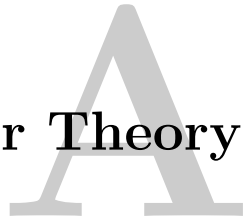
References

- [Ang26] I. Angelo, *Implementation of (Somewhat) Practical Anonymous Router*, version 1.0.0, Jul. 2026. [Online]. Available: <https://github.com/ImreAngelo/sPAR>.
- [APS15] M. R. Albrecht, R. Player, and S. Scott, *On the concrete hardness of learning with errors*, Cryptology ePrint Archive, Paper 2015/046, 2015. [Online]. Available: <https://eprint.iacr.org/2015/046>.
- [BAB+22] A. A. Badawi, A. Alexandru, *et al.*, *OpenFHE: Open-source fully homomorphic encryption library*, Cryptology ePrint Archive, Paper 2022/915, 2022. [Online]. Available: <https://eprint.iacr.org/2022/915>.
- [BEHZ16] J.-C. Bajard, J. Eynard, *et al.*, *A full RNS variant of FV like somewhat homomorphic encryption schemes*, Cryptology ePrint Archive, Paper 2016/510, 2016. [Online]. Available: <https://eprint.iacr.org/2016/510>.
- [BGV11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, *Fully homomorphic encryption without bootstrapping*, Cryptology ePrint Archive, Paper 2011/277, 2011. [Online]. Available: <https://eprint.iacr.org/2011/277>.
- [Bra12] Z. Brakerski, *Fully homomorphic encryption without modulus switching from classical GapSVP*, Cryptology ePrint Archive, Paper 2012/078, 2012. [Online]. Available: <https://eprint.iacr.org/2012/078>.
- [BV11] Z. Brakerski and V. Vaikuntanathan, *Efficient fully homomorphic encryption from (standard) LWE*, Cryptology ePrint Archive, Paper 2011/344, 2011. [Online]. Available: <https://eprint.iacr.org/2011/344>.
- [CCR19] H. Chen, I. Chillotti, and L. Ren, “Onion ring oram: Efficient constant bandwidth oblivious ram from (leveled) tfhe”, in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’19, London, United Kingdom: Association for Computing Machinery, 2019, pp. 345–360. [Online]. Available: <https://doi.org/10.1145/3319535.3354226>.
- [CGGI18] I. Chillotti, N. Gama, *et al.*, *TFHE: Fast fully homomorphic encryption over the torus*, Cryptology ePrint Archive, Paper 2018/421, 2018. [Online]. Available: <https://eprint.iacr.org/2018/421>.
- [DM14] L. Ducas and D. Micciancio, *FHEW: Bootstrapping homomorphic encryption in less than a second*, Cryptology ePrint Archive, Paper 2014/816, 2014. [Online]. Available: <https://eprint.iacr.org/2014/816>.

- [DP25] D. Das and J. Park, *sPAR: (somewhat) practical anonymous router*, Cryptology ePrint Archive, Paper 2025/860, 2025. [Online]. Available: <https://eprint.iacr.org/2025/860>.
- [Gen09] C. Gentry, “A fully homomorphic encryption scheme”, Ph.D. dissertation, Stanford University, 2009. [Online]. Available: <https://crypto.stanford.edu/craig>.
- [GHS12] C. Gentry, S. Halevi, and N. P. Smart, *Homomorphic evaluation of the AES circuit*, Cryptology ePrint Archive, Paper 2012/099, 2012. [Online]. Available: <https://eprint.iacr.org/2012/099>.
- [GSW13] C. Gentry, A. Sahai, and B. Waters, *Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based*, Cryptology ePrint Archive, Paper 2013/340, 2013. [Online]. Available: <https://eprint.iacr.org/2013/340>.
- [HPS18] S. Halevi, Y. Polyakov, and V. Shoup, *An improved RNS variant of the BFV homomorphic encryption scheme*, Cryptology ePrint Archive, Paper 2018/117, 2018. [Online]. Available: <https://eprint.iacr.org/2018/117>.
- [KPZ21] A. Kim, Y. Polyakov, and V. Zucca, *Revisiting homomorphic encryption schemes for finite fields*, Cryptology ePrint Archive, Paper 2021/204, 2021. [Online]. Available: <https://eprint.iacr.org/2021/204>.
- [LY25] K. H. Lee and J. W. Yoon, *Homomorphic field trace revisited : Breaking the cubic noise barrier*, Cryptology ePrint Archive, Paper 2025/1088, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1088>.
- [RAD78] R. L. Rivest, L. Adleman, and M. L. Dertouzos, “On data banks and privacy homomorphisms”, in *Foundations of Secure Computation*, 1978, pp. 169–180.
- [SEV+15] Y. Sun, A. Edmundson, *et al.*, “RAPTOR: Routing attacks on privacy in tor”, in *24th USENIX Security Symposium (USENIX Security 15)*, Washington, D.C.: USENIX Association, Aug. 2015, pp. 271–286. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/sun>.
- [SV11] N. P. Smart and F. Vercauteren, *Fully homomorphic SIMD operations*, Cryptology ePrint Archive, Paper 2011/133, 2011. [Online]. Available: <https://eprint.iacr.org/2011/133>.
- [SW21] E. Shi and K. Wu, *Non-interactive anonymous router*, Cryptology ePrint Archive, Paper 2021/435, 2021. [Online]. Available: <https://eprint.iacr.org/2021/435>.

Appendix

Number Theory



A.1 Chinese Remainder Theorem

Could be stated here just to be more complete

A.2 Number Theoretic Transform

- Turns a convolution into a nega/cyclic
- Same as FFT but over different ring